

CampusClub360

Phase 5: Apex Programming (Developer)

Apex Classes:

- **AttendanceController**

- Purpose: Manage attendance records through LWC (QR scan integration).
- Key Functions: Create attendance, fetch student details, validate duplicates.

```
public with sharing class AttendanceController {  
    // Wrapper to return attendance record + flag whether it was newly created  
    public class AttendanceResult {  
        @AuraEnabled public Attendance__c attendance;  
        @AuraEnabled public Boolean isNew;  
        public AttendanceResult(Attendance__c a, Boolean n) {  
            this.attendance = a;  
            this.isNew = n;  
        }  
    }  
}
```

```
@AuraEnabled(cacheable=true)  
public static List<Club_Meeting__c> getMeetings() {  
    return [  
        SELECT Id, Name, Meeting_Date__c  
        FROM Club_Meeting__c  
        ORDER BY Meeting_Date__c ASC  
    ];  
}
```

```
@AuraEnabled(cacheable=true)  
public static Student__c findStudentByBarcode(String barcodeValue) {  
    List<Student__c> s = [  
        SELECT Id, Name, Student_ID__c, Barcode_Value__c  
        FROM Student__c  
        WHERE Barcode_Value__c = :barcodeValue  
        LIMIT 1  
    ];  
    return s.isEmpty() ? null : s[0];  
}
```

```
@AuraEnabled
```

```

public static AttendanceResult createAttendance(Id studentId, Id meetingId) {
    // Attempt to find existing attendance
    List<Attendance__c> existing = [
        SELECT Id, Check_In_Time__c, Student__r.Name, Club_Meeting__r.Name
        FROM Attendance__c
        WHERE Student__c = :studentId AND Club_Meeting__c = :meetingId
        LIMIT 1
    ];
    if (!existing.isEmpty()) {
        return new AttendanceResult(existing[0], false);
    }

    // Create new Attendance (also populate UniqueKey__c if you created it)
    Attendance__c a = new Attendance__c(
        Student__c = studentId,
        Club_Meeting__c = meetingId,
        Check_In_Time__c = System.now()
    );

    // If you have a UniqueKey__c text unique field, set it here:
    // a.UniqueKey__c = studentId + '-' + meetingId;

    insert a;

    // Re-query to include relationship fields for the UI
    a = [
        SELECT Id, Check_In_Time__c, Student__r.Name, Club_Meeting__r.Name
        FROM Attendance__c
        WHERE Id = :a.Id
        LIMIT 1
    ];

    return new AttendanceResult(a, true);
}
}

```

- **SportsRegistrationController:**

- Purpose: Handle sports team registrations.
- Key Functions: Save team details on Sports Registration Teams

```

public with sharing class SportsRegistrationController {
    @AuraEnabled

```

```

    public static void createRegistration(String sport, String teamName, Integer players,
    Decimal fee) {
        Sports_Registration__c reg = new Sports_Registration__c();
        reg.Name = teamName; // use standard Name
        reg.Sport__c = sport;
        reg.Number_of_Players__c = players;
        reg.Registration_Fee__c = fee;
        insert reg;
    }
}

```

- **SportsItemController**

- Purpose: Manage inventory of store items.
- Key Functions: Retrieve items, update stock, prevent duplicate barcodes.

```

public with sharing class SportsItemController {

    @AuraEnabled(cacheable=true)
    public static Sports_Item__c getItemByBarcode(String barcode) {
        try {
            List<Sports_Item__c> items = [
                SELECT Id, Name, Price__c, Barcode__c, Stock_Quantity__c
                FROM Sports_Item__c
                WHERE Barcode__c = :barcode
                LIMIT 1
            ];
            if (!items.isEmpty()) {
                return items[0];
            }
            return null;
        } catch (Exception e) {
            throw new AuraHandledException('Error fetching item: ' + e.getMessage());
        }
    }

    // Helper inner class to receive cart data from LWC
    public class CartItem {
        @AuraEnabled public String id;
        @AuraEnabled public String name;
        @AuraEnabled public Decimal price;
        @AuraEnabled public String barcode;
        @AuraEnabled public Integer quantity;
    }
}

```

```

@AuraEnabled
public static String finalizeSale(List<CartItem> items, Decimal totalAmount) {
    if (items == null || items.isEmpty()) {
        throw new AuraHandledException('Cart is empty. Nothing to finalize.');
```

```

    }

    try {
        // Create Sale record
        Sports_Sale__c sale = new Sports_Sale__c();
        sale.Name = 'Sale - ' + DateTime.now().format('yyyy-MM-dd HH:mm');
        sale.Sale_Date__c = System.now();
        sale.Total_Amount__c = totalAmount;
        insert sale;

        // Build item summary text
        String summary = "";
        List<Sports_Item__c> updates = new List<Sports_Item__c>();

        for (CartItem ci : items) {
            summary += ci.name + ' (x' + ci.quantity + ') - ₹' + (ci.price * ci.quantity) +
'\n';

            if (ci.id != null) {
                // Try to fetch by Id
                List<Sports_Item__c> results = [
                    SELECT Id, Stock_Quantity__c
                    FROM Sports_Item__c
                    WHERE Id = :ci.id
                    LIMIT 1
                ];
                if (!results.isEmpty()) {
                    Sports_Item__c item = results[0];
                    Integer currentStock = item.Stock_Quantity__c == null ? 0 :
Integer.valueOf(item.Stock_Quantity__c);
                    item.Stock_Quantity__c = Math.max(0, currentStock - ci.quantity);
                    updates.add(item);
                }
            }
        }

        if (!updates.isEmpty()) {
            update updates;
        }
    }
}

```

```

    }

    // Update sale with summary
    sale.Items_Summary__c = summary;
    update sale;

    return 'Sale finalized successfully. Id: ' + sale.Id;
} catch (Exception e) {
    throw new AuraHandledException('Failed to finalize sale: ' + e.getMessage());
}
}
}

```

- **SportsSaleController:**

- Purpose: Manage sales transactions in Campus Mart.
- Key Functions: Create sales records, calculate total amount, integrate with cart.

```

public with sharing class SportsSaleController {

    // Wrapper to map cart items coming from LWC
    public class CartItemWrapper {
        @AuraEnabled public String id;
        @AuraEnabled public String name;
        @AuraEnabled public Decimal price;
        @AuraEnabled public String barcode;
        @AuraEnabled public Integer quantity;
    }

    @AuraEnabled
    public static Id finalizeSale(String itemsJson, Decimal totalAmount) {
        if (String.isBlank(itemsJson)) {
            throw new AuraHandledException('Cart is empty. Cannot finalize sale.');
```

```

// Build summary for Sale record
String summary = "";
for (CartItemWrapper item : items) {
    summary += item.name + ' (x' + item.quantity + ') - ₹' +
        String.valueOf(item.price * item.quantity) + '\n';
}

// Create Sale record
Sports_Sale__c sale = new Sports_Sale__c();
sale.Name = 'Sale - ' + DateTime.now().format('yyyy-MM-dd HH:mm');
sale.Sale_Date__c = System.now();
sale.Total_Amount__c = totalAmount;
sale.Items_Summary__c = summary;
insert sale;

// Reduce stock
List<Sports_Item__c> updates = new List<Sports_Item__c>();
for (CartItemWrapper item : items) {
    Sports_Item__c dblItem = [
        SELECT Id, Stock_Quantity__c
        FROM Sports_Item__c
        WHERE Id = :item.id
        LIMIT 1
    ];
    if (dblItem.Stock_Quantity__c >= item.quantity) {
        dblItem.Stock_Quantity__c -= item.quantity;
        updates.add(dblItem);
    } else {
        throw new AuraHandledException(
            'Not enough stock for item: ' + item.name
        );
    }
}
if (!updates.isEmpty()) {
    update updates;
}

return sale.Id;
}
}

```

2. Scheduler Apex:

- **DailyMeetingCleanupScheduler:**

- Purpose: Schedule a job to clean old club meeting data automatically.
- Key Functions: Executes scheduled jobs daily to trigger batch cleanup.

```
// Scheduler class
global class DailyMeetingCleanupScheduler implements Schedulable {
    global void execute(SchedulableContext sc) {
        DailyMeetingCleanupBatch batch = new DailyMeetingCleanupBatch();
        Database.executeBatch(batch, 50); // batch size 50
    }
}
```

3. Batch Apex:

- **DailyMeetingCleanupBatch:**

- Purpose: Delete expired or past club meetings in bulk.
- Key Functions: Query old records, delete in batches for performance.

```
// Batch class
global class DailyMeetingCleanupBatch implements Database.Batchable<SObject>,
Database.Stateful {

    global Database.QueryLocator start(Database.BatchableContext bc) {
        Date yesterday = Date.today().addDays(-1);
        return Database.getQueryLocator([
            SELECT Id, Name, CreatedBy.Email, CreatedBy.Name
            FROM Club_Meeting__c
            WHERE Meeting_Date__c = :yesterday
        ]);
    }

    global void execute(Database.BatchableContext bc, List<Club_Meeting__c>
meetings) {
        for (Club_Meeting__c meeting : meetings) {

            // Get attendance for this meeting
            List<Attendance__c> attendanceList = [
                SELECT Id, Student__r.Name, Student_Email__c
                FROM Attendance__c
                WHERE Club_Meeting__c = :meeting.Id
            ];

            if (!attendanceList.isEmpty()) {

                // Build HTML table
                String htmlTable = '<table border="1" style="border-collapse:collapse;">' +
```

```

        '<tr><th>Student Name</th><th>Email</th></tr>';
        for (Attendance__c att : attendanceList) {
            String studentName = (att.Student__r != null) ? att.Student__r.Name :
'Unknown';
            String studentEmail = (att.Student_Email__c != null) ?
att.Student_Email__c : 'N/A';
            htmlTable += '<tr><td>' + studentName + '</td><td>' + studentEmail +
'</td></tr>';
        }
        htmlTable += '</table>';

        // Send email to meeting creator
        Messaging.SingleEmailMessage mail = new
Messaging.SingleEmailMessage();
        mail.setToAddresses(new String[] { meeting.CreatedBy.Email });
        mail.setSubject('Attendance Report for ' + meeting.Name);
        mail.setHtmlBody('Dear ' + meeting.CreatedBy.Name + ',<br/><br/>' +
            'Here is the attendance for your meeting: ' + meeting.Name +
'<br/><br/>' +
            htmlTable +
            '<br/><br/>Regards,<br/>System');

        Messaging.sendEmail(new Messaging.SingleEmailMessage[] { mail });

        // Delete attendance
        delete attendanceList;
    }
}

global void finish(Database.BatchableContext bc) {
    // Optional logging
}
}

```

4. Apex Triggers:

- **AttendanceTrigger:**

- **Purpose:** Auto-create attendance records when a student is scanned.
- **Business Hours Logic:** Ensures attendance is only recorded if the scan happens within the **defined Club Hours**
- **Key Logic:**
 - On insert → check current time vs. club hours.
 - If outside hours → throw error or block attendance.


```

trigger AttendanceTrigger on Attendance__c (before insert, before update) {
    // Query Club Business Hours once
    BusinessHours clubHours = [
        SELECT Id
        FROM BusinessHours
        WHERE Name = 'Club Hours'
        LIMIT 1
    ];

    for (Attendance__c att : Trigger.new) {
        // Skip if no check-in time provided
        if (att.Check_In_Time__c == null) continue;

        // Check if inside Club business hours
        Boolean isValid = BusinessHours.isWithin(
            clubHours.Id,
            att.Check_In_Time__c
        );

        if (!isValid) {
            att.Check_In_Time__c.addError(
                'Attendance not allowed outside Club timings (10 AM – 3 PM, Mon–Sat,
                excluding holidays).'
            );
        }
    }
}

```

- **SportsSaleTrigger:**

- **Purpose:** Manage stock updates when an item is sold.
- **Business Hours Logic:** Ensures sales can only be recorded during defined Store Hours.
- **Key Logic:** On insert → check sale time vs. store hours.

```

trigger SportsSaleTrigger on Sports_Sale__c (before insert, before update) {
    // Query Store Business Hours once
    BusinessHours storeHours = [
        SELECT Id
        FROM BusinessHours
        WHERE Name = 'Store Hours'
        LIMIT 1
    ];
}

```

```

for (Sports_Sale__c sale : Trigger.new) {
    // Skip if no sale date provided
    if (sale.Sale_Date__c == null) continue;

    // Check if inside Store business hours
    Boolean isValid = BusinessHours.isWithin(
        storeHours.Id,
        sale.Sale_Date__c
    );

    if (!isValid) {
        sale.Sale_Date__c.addError(
            'Store purchase not allowed outside Store timings (10 AM – 7 PM, Mon–Sat,
            excluding Sundays/holidays).'
        );
    }
}
}

```

5. Test classes:

- **AttendanceTriggerTest**

```

@isTest
public class AttendanceTriggerTest {

    @testSetup
    static void setupData() {
        // Create a sample student
        Student__c s = new Student__c(Name = 'Test Student');
        insert s;

        // Create a sample Club Meeting
        Club_Meeting__c m = new Club_Meeting__c(Name = 'Weekly Meet');
        insert m;
    }

    @isTest
    static void testValidAttendance() {
        // Get IDs
        Student__c s = [SELECT Id FROM Student__c LIMIT 1];
        Club_Meeting__c m = [SELECT Id FROM Club_Meeting__c LIMIT 1];

        // Valid Check-In (weekday, within 10 AM – 3 PM)
    }
}

```

```

        Attendance__c att = new Attendance__c(
            Name = 'Valid Attendance',
            Student__c = s.Id,
            Club_Meeting__c = m.Id,
            Check_In_Time__c = Datetime.newInstance(2025, 9, 18, 11, 0, 0) // Thu 11
AM
        );
        Test.startTest();
        insert att;
        Test.stopTest();

        System.assertNotEquals(null, att.Id, 'Attendance should be saved inside business
hours');
    }

    @isTest
    static void testInvalidAttendance() {
        Student__c s = [SELECT Id FROM Student__c LIMIT 1];
        Club_Meeting__c m = [SELECT Id FROM Club_Meeting__c LIMIT 1];

        // Invalid Check-In (weekday, but 5 PM = outside Club Hours)
        Attendance__c att = new Attendance__c(
            Name = 'Invalid Attendance',
            Student__c = s.Id,
            Club_Meeting__c = m.Id,
            Check_In_Time__c = Datetime.newInstance(2025, 9, 18, 17, 0, 0) // Thu 5 PM
        );

        Test.startTest();
        try {
            insert att;
            System.assert(false, 'Expected error for attendance outside hours');
        } catch (DmlException e) {
            System.assert(e.getMessage().contains('Attendance not allowed'), 'Error
should mention attendance hours');
        }
        Test.stopTest();
    }
}

```

- **SportsSaleTriggerTest:**

```

@isTest

```

```

public class SportsSaleTriggerTest {

    @testSetup
    static void setupData() {
        // Create a sample Sports Item
        Sports_Item__c item = new Sports_Item__c(
            Name = 'Football',
            Barcode__c = 'FB123',
            Price__c = 500,
            Stock_Quantity__c = 10
        );
        insert item;
    }

    @isTest
    static void testValidSale() {
        Sports_Item__c item = [SELECT Id, Price__c FROM Sports_Item__c LIMIT 1];

        Sports_Sale__c sale = new Sports_Sale__c(
            Name = 'Valid Sale',
            Sale_Date__c = Datetime.newInstance(2025, 9, 20, 18, 0, 0), // Sat 6 PM
            Total_Amount__c = item.Price__c,
            Items_Summary__c = 'Football x1'
        );

        Test.startTest();
        insert sale;
        Test.stopTest();

        System.assertNotEquals(null, sale.Id, 'Sale should be saved inside business
hours');
    }

    @isTest
    static void testInvalidSale() {
        Sports_Item__c item = [SELECT Id, Price__c FROM Sports_Item__c LIMIT 1];

        Sports_Sale__c sale = new Sports_Sale__c(
            Name = 'Invalid Sale',
            Sale_Date__c = Datetime.newInstance(2025, 9, 21, 11, 0, 0), // Sunday 11 AM
            Total_Amount__c = item.Price__c,
            Items_Summary__c = 'Football x1'
        );
    }
}

```

```
Test.startTest();
try {
    insert sale;
    System.assert(false, 'Expected error for sale outside hours');
} catch (DmlException e) {
    System.assert(e.getMessage().contains('Store purchase not allowed'), 'Error
should mention store hours');
}
Test.stopTest();
}
```